L18: Beyond Simple Backpropagation

C. V. Jawahar

IIIT Hyderabad

March 26, 2026

Backpropagation:Recap

- **Step 0:** Initialize the Network (MLP), weights, W^0.
- **Step 1:** Do forward pass for a mini batch (B) of randomly selected samples. Predict outputs with the existing weights.

$$o_i = f(\mathbf{W}, x_i)$$

- **Step 2:** Compute Loss for the set of samples.

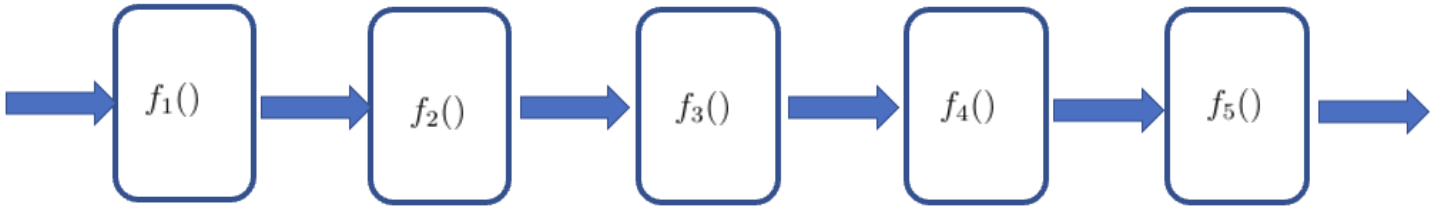
$$L = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}(o_i, y_i)$$

- **Step 3:** Update all the weights using gradient descent.

$$W^{k+1} \leftarrow W^k - \eta \frac{\partial L}{\partial W^k}$$

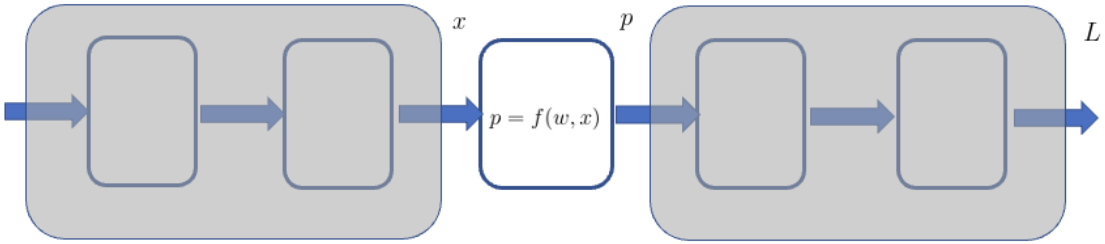
- **Step 4:** Repeat all steps (until the loss is acceptable.)

Notes: Recap



- Two major steps (i) Forward Pass (ii) Backward Pass.
- If we compute gradients backward, it is efficient. "We propagate gradients backwards."
- Some blocks in the chain/graph may have learnable parameter and some may not have.
- Softmax, Activations, Loss etc. can be seen as layers (or some computational functions/blocks in the network).
- Long chain of multiplications can lead to "Vanishing gradients" or "exploding gradients".

Notes: Recap



- For every layer we use, we need to know how to compute derivative of output wrt input (i.e., $\frac{\partial p}{\partial x}$) so that gradients can be propagated backward.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial x}$$

- For every layer, we need to know how to compute derivative of output wrt parameters/weights (i.e., $\frac{\partial p}{\partial w}$), (if we have) so that weights can be updated or we can train/learn.

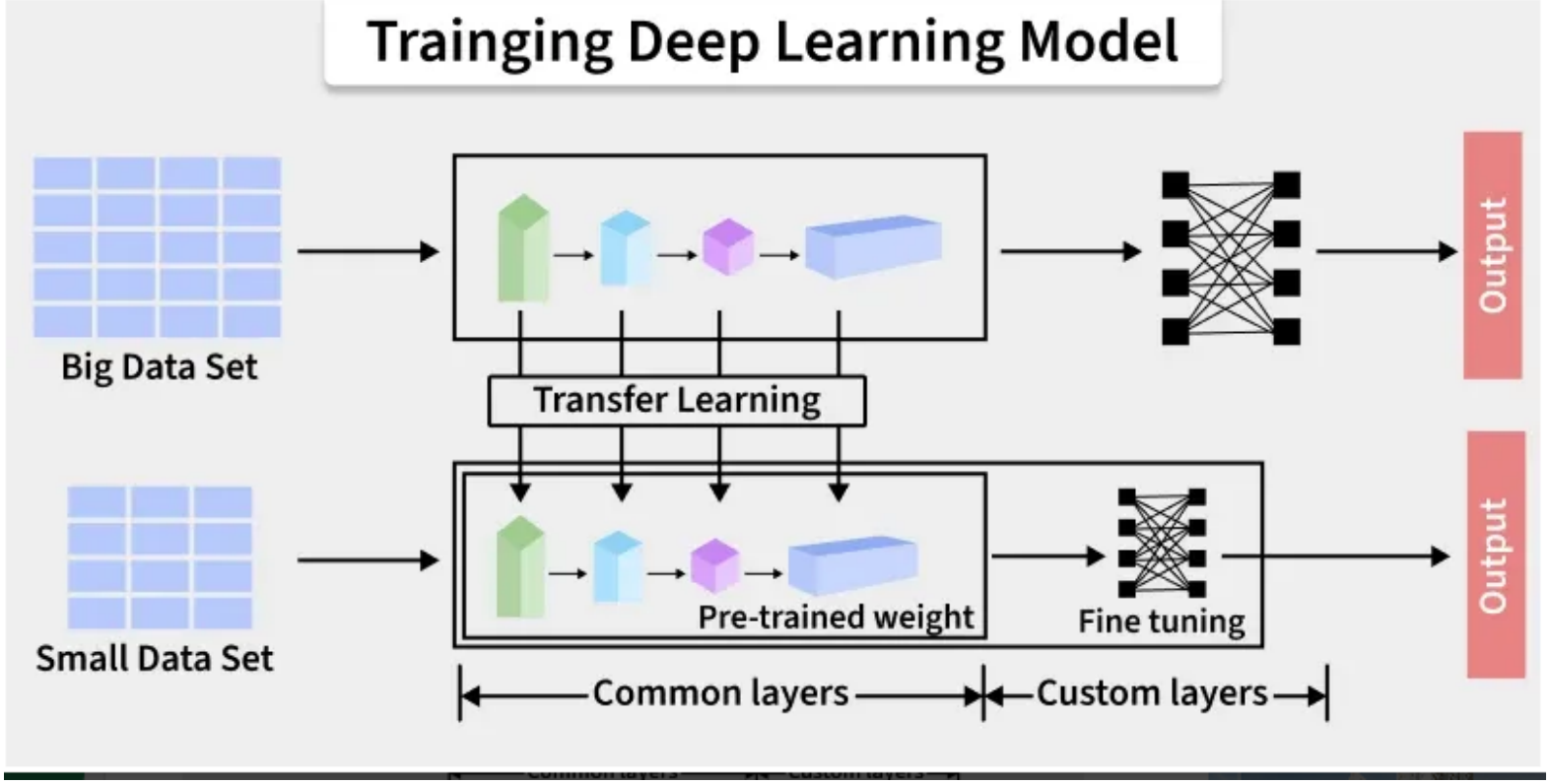
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial w}$$

Recap: Opportunities over Simple BP

- Better Initialization
- Effective Gradient Estimates
- Better Update Rule: Momentum
- Use of Subgradients [1]
- Choice of Optimizer
- Loss Function
- Termination
- More ?

[1] https://medium.com/@dassandipan9080/when-gradients-fail-subgradient-descent-and-escaping-saddle-points-in-opti

PreTrain as Initialization



Appreciate PreTraining

- PreTrain and Fine Tune
- PreTrain as "Initialization"
- Pretrain as "Transfer Learning"
- Pretrain on a related dataset
- Pretrain on a related "task"
- What do we reuse? Early layers
- Pretrain with self/un-supervision (eg. Autoencoder; next word prediction)

(Batch) Normalization

- We normalize the input data before feeding to the NNs
- What about the input distribution at each hidden layer? the distribution changes with iterations internal co-variate shift
- Solution: Normalize the layer's input over a mini-batch

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

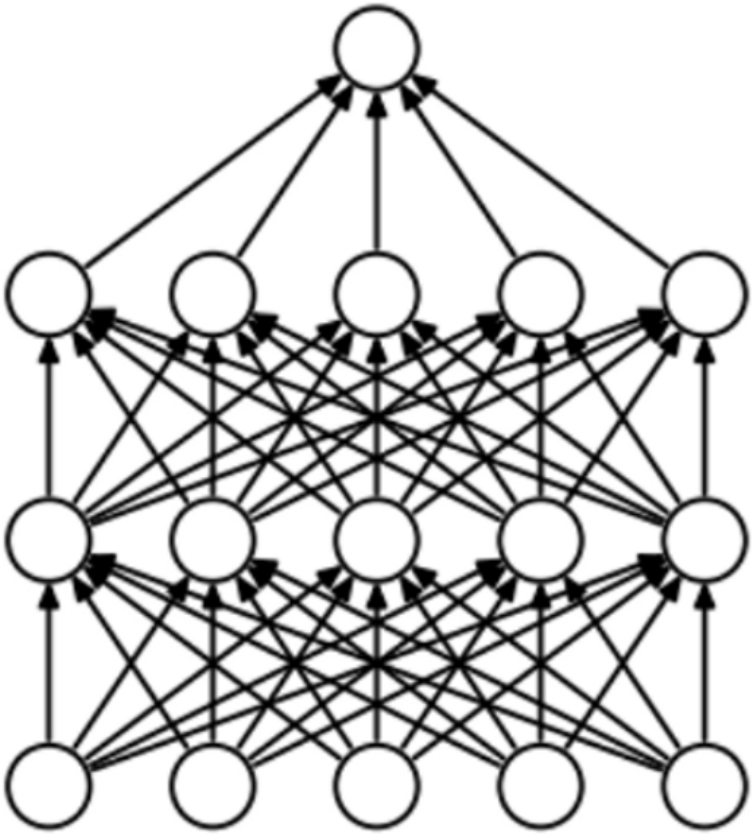
$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

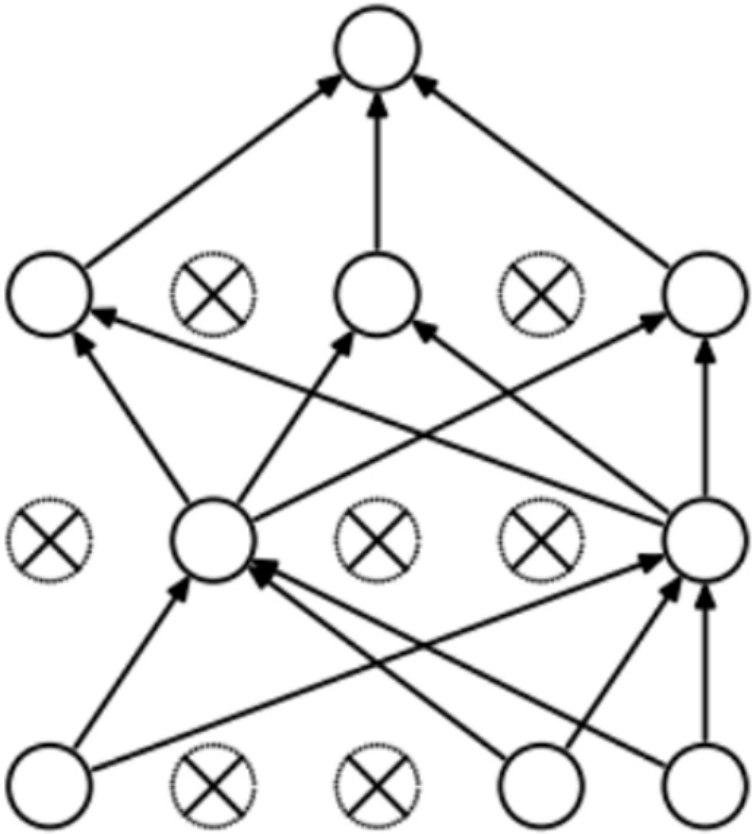
Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Q: What do we do at test time?

DropOut



(a) Standard Neural Net



(b) After applying dropout.

DropOut

- Ignore/delete/mask certain neurons while training.
  - Get a simpler network. Eg. Multiply the outputs by 0 or 1 at random.
  - Equivalent to creating many neural networks.
- Reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons.
- Force to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.
- Set the output of each hidden neuron to zero with a probability of p (say 0.5).
- The neurons which are "dropped out" in this way do not contribute to the forward pass and do not participate in back propagation.
- Thus neural network samples a different architecture, but all these architectures share weights.
- At test time, scale outputs by probability p.

Remember Taylor Series? (Rewriting ..)

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \dots$$

$$f(y) = f(x) + (y-x)f'(x) + \frac{(y-x)^2}{2}f''(x) + \dots$$

$$f(\mathbf{y}) = f(\mathbf{x}) + [\mathbf{y}-\mathbf{x}] \cdot \nabla \mathbf{f}(\mathbf{x}) + \frac{1}{2}[\mathbf{y}-\mathbf{x}]^T \mathbf{H}(\mathbf{x})[\mathbf{y}-\mathbf{x}] + \dots$$

$$\begin{aligned} f(\mathbf{w^{k+1}}) &= f(\mathbf{w^k}) + [\mathbf{w^{k+1}} - \mathbf{w^k}] \cdot \nabla f(\mathbf{w^k}) \\ &\quad + \frac{1}{2}[\mathbf{w^{k+1}} - \mathbf{w^k}]^T \mathbf{H}[\mathbf{w^{k+1}} - \mathbf{w^k}] + \dots \end{aligned}$$

$$\text{If } \mathbf{w}^{k+1} \leftarrow \mathbf{w}^k + \mathbf{s} \tag{1}$$

$$f(\mathbf{w^{k+1}}) = f(\mathbf{w^k}) + \mathbf{s}^T \nabla f + \frac{1}{2}\mathbf{s}^T \mathbf{H}\mathbf{s} + \dots$$

Note that we ∇ and Hessians are evaluated at $\mathbf{w}^k$, It is not written here explicitly in some of these equations, to minimize the clutter.

GD Minimization of $J()$

$$
\begin{aligned}
J(\mathbf{w^{k+1}}) &= J(\mathbf{w^k}) + [\mathbf{w^{k+1}} - \mathbf{w^k}] \cdot \nabla J(\mathbf{w^k}) \\
&\quad + \frac{1}{2}[\mathbf{w^{k+1}} - \mathbf{w^k}]^T \mathbf{H} [\mathbf{w^{k+1}} - \mathbf{w^k}] + \ldots
\end{aligned}
$$

$$
J(\mathbf{w^{k+1}}) = J(\mathbf{w^k}) + \mathbf{s}^T \nabla J + \frac{1}{2}\mathbf{s}^T \mathbf{H} \mathbf{s} + \ldots
$$

Why do GD works?: Ans: non increasing updates

Let us remember our update rule as

$$\mathbf{w^{k+1}} = w^k - \eta \nabla J(\mathbf{w}^k)$$

or the change in weight (or weight uodate)

$$\mathbf{w}^{k+1} - \mathbf{w}^k = \mathbf{s} = -\eta \nabla J(\mathbf{w}^k)$$

Also let us look at the first order approximation of the Taylor series as:

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w}^k) + \mathbf{s}^T \nabla J(\mathbf{w}^k)$$

Substituting for $\mathbf{s}$

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w}^k) - \eta \nabla J(\mathbf{w}^k)^T \nabla J(\mathbf{w}^k)$$

or

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w}^k) - \eta((\text{a non negative quantity})$$

Therefore, when η is positive, we can argue that the objective is improving (non-increasing) in every iteration. When the gradient is zero, no change in the objective and the iterations stops.

Is there an Optimal Learning Rate?

What is the best learning rate η?

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w^k}) + \mathbf{s}^T \nabla J + \frac{1}{2}\mathbf{s}^T \mathbf{H}\mathbf{s}$$

$$\mathbf{w}^{k+1} - \mathbf{w}^k = \mathbf{s} = -\eta \nabla J(\mathbf{w}^k)$$

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w^k}) - \eta \nabla \mathbf{J^T} \nabla \mathbf{J} + \frac{\eta^2}{2} \nabla \mathbf{J}^T \mathbf{H} \nabla \mathbf{J}$$

We want to optimize for η by differentiating with respect to η and equating to zero.

$$-||\nabla \mathbf{J}||^2 + \eta \nabla \mathbf{J}^T \mathbf{H} \nabla \mathbf{J} = 0$$

$$\eta = \frac{||\nabla \mathbf{J}||^2}{\nabla \mathbf{J}^T \mathbf{H} \nabla \mathbf{J}}$$

Note that this needs change at every iteration k since J and $\mathbf{H}$ are also function of $\mathbf{w^k}$.

Recap: Gradient, Hessian and Jacobian

Gradient $f : R^n \to R$

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \cdot \\ \cdot \\ \cdot \frac{\partial f}{\partial x_n} \end{bmatrix}$$

You may know Hessian: $f : R^n \to R$

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 f}{x_1^2} & \frac{\partial^2 f}{x_1 x_2} & \cdots & \frac{\partial^2 f}{x_1 x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial^2 f}{x_n x_1} & \frac{\partial^2 f}{x_n x_2} & \cdots & \frac{\partial^2 f}{x_n^2} \end{bmatrix}$$

Alsop Jacobian of $f : R^n \to R^m$

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Why Steepest Gradient?

We know from the first order truncated taylor series expansion that

$$J(\mathbf{x} + \Delta\mathbf{x}) = J(\mathbf{x}) + \Delta\mathbf{x}^T \nabla J(\mathbf{x})$$

The directional derivative of ∇J in terms of any direction $\mathbf{u}$ is $\mathbf{u}^T \nabla J$. Our objective is to find what should be the best direction. This leads to the new problem:

$$\min_{\mathbf{u};||\mathbf{u}=1} \mathbf{u}^T \nabla J$$

$$= \min_{\mathbf{u};||\mathbf{u}=1} ||\mathbf{u}|| \cdot ||\nabla J|| \cos\theta$$

This quantity is minized when $\mathbf{u}$ is pointing in the opposite direction of ∇J and $\cos\theta = -1$. Therefore

$$\mathbf{x}^{new} = \mathbf{x}^{old} - \eta \nabla J$$

Better Update Rule: Second Order Methods

Let us also see if there is a better update rule. i.e., is there a better $\mathbf{s}$?
Let us revisit:

$$J(\mathbf{w^{k+1}}) = J(\mathbf{w^k}) + \mathbf{s}^T \nabla J + \frac{1}{2}\mathbf{s}^T \mathbf{H}\mathbf{s}$$

Differentiating with respect to $\mathbf{s}$ and equating to zero:

$$\nabla \mathbf{J} + \mathbf{H}\mathbf{s} = 0$$

Or

$$\mathbf{s} = -\mathbf{H}^{-1}\nabla \mathbf{J}$$

This leads to a better update rule known as Newton's updates.
Newton's method is known to provide faster convergence for the optimization. Indeed the second order approximations we use here is more accurrate than the first order one.

Second Order Methods (Newton's)

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \mathbf{H}^{-1}\nabla J$$

- Gradient gives you direction; Hessian tells us how much to trust that direction
- Very fast convergence, often quadratic convergence
- When the Hessian has negative eigenvalues (eg. saddle points), then steps along the corresponding eigenvectors are gradient ascent steps. To counteract this, regularize:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - (\mathbf{H} + \alpha I)^{-1}\nabla J$$

- As α becomes larger, this turns into first order gradient descent with learning rate $\frac{1}{\alpha}$.
- Hessian is $d \times d$, memory requirement is high. Also inversion is costly.

Optimizer: Adam (Advanced)

- **Simple GD** $w^{t+1} \leftarrow w^t - \eta g_t$ where $g_t = \nabla J(w^k)$
- **Momentum:** smooths gradients; builds velocity; reduses zig-zag

$$w^{t+1} \leftarrow w^t - \eta m_t \text{ where } m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

  momentum is an exponential moving average of gradients
- **Adaptive learning rates** Some parameters need large rates while some small. Large gradients need small step; small gradients need large step.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$w^{t+1} = w^t - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}$$

- **Adam** Momentum + Adaptive Scaling (and Bias correction)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$w^{t+1} = w^t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Optimizer: Adam (Advanced)

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
  $m_0 \leftarrow 0$ (Initialize 1st moment vector)
  $v_0 \leftarrow 0$ (Initialize 2nd moment vector)
  $t \leftarrow 0$ (Initialize timestep)
  while θ_t not converged **do**
    $t \leftarrow t + 1$
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
    $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
    $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
    $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
    $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
    $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
  end while
  return θ_t (Resulting parameters)

QUIZ-II

Scope:

- Overview of Neural Networks
- Neuron Models
- Single Layer Perceptron
- Multi Layer Perceptron
- Gradient Descent
- Backpropagation
- Second Order Methods